

CA - 2

Name: Ritul Kumar

Stream: CSE(DS)

Uni. Roll No: 10930524002

Class Roll No.: 02

Subject: DAA(PCC-CS494)

1.) Write the algorithm of merge sort and analyse its time complexity.

→ Merge Sort is a Divide and Conquer algorithm. It keeps breaking the array into smaller parts until only single elements remain, and then merges them back in sorted order.

Algorithm :

MERGE-SORT(A, low, high):

1.) If $low < high$

2.) Find $mid = (low + high) / 2$

3.) Call MERGE-SORT(A, low, mid)

4.) Call MERGE-SORT(A, mid+1, high)

5.) Call MERGE(A, low, mid, high)

MERGE(A, low, mid, high):

1.) Create two temporary arrays

- Left = A[low...mid]

- Right = A[mid+1...high]

2.) Compare elements of Left and Right

3.) Insert smaller element into original array.

4.) Repeat until all elements are merged.

Time Complexity Analysis:-

Merge Sort follows this recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Now, dividing $\rightarrow \log n$ levels

Merging $\rightarrow$ takes n time per level

So, total time: $T(n) = O(n \log n)$

- Best case: $O(n \log n)$
- Average case: $O(n \log n)$
- Worst case: $O(n \log n)$

2) Solve using Substitution Method.

(a) $T(n) = 1$ when $n = 1$

$T(n) = T(n/2) + 1$ when $n \geq 1$

$\rightarrow T(n) = T(n/2) + 1$

$$= T(n/4) + 1 + 1$$

$$= T(n/8) + 3$$

after k steps: $T(n) = T(n/2^k) + k$

when $n/2^k = 1$

$$k = \log_2 n$$

$$T(n) = T(1) + \log n \quad (\text{putting } (n/2^k) = 1)$$

$$T(n) = 1 + \log n \quad (\because T(1) = 1)$$

$$\therefore \text{Complexity} = T(n) = O(\log n)$$

$$2 \rightarrow (b) \quad T(n) = 1 \quad \text{when } n = 1.$$

$$T(n) = T(n/2) + 1 \quad \text{when } n > 1$$

$$\Rightarrow T(n) = n + T(n/2)$$

$$= n + n/2 + T(n/4)$$

$$= n + n/2 + n/4 + \dots + T(n/2^k) \quad \left\{ \begin{array}{l} \text{After } k \\ \text{steps} \end{array} \right\}$$

$$\text{when } n/2^k = 1$$

$$k = \log n$$

$$T(n) = n(1 + 1/2 + 1/4 + \dots)$$

$$= n \left(\frac{1}{1 - 1/2} \right)$$

$$\left\{ \begin{array}{l} \text{Using G.P.} \\ S_n = \frac{a}{1 - r} \end{array} \right\}$$

$$= 2n$$

$$\therefore \text{Complexity : } T(n) = O(n)$$

$$2. \rightarrow (c) T(n) = 1 \quad \text{when } n = 0$$

$$T(n) = 2T(n-1) + 1 \quad \text{when } n > 0$$

$$\rightarrow T(n) = 2T(n-1) + 1$$

$$= 2(2T(n-2) + 1) + 1$$

$$= 4T(n-2) + 3 = 2^2 T(n-2) + (2^2 - 1)$$

$$= 8T(n-3) + 7 = 2^3 (T(n-3) + (2^3 - 1))$$

$$\text{So, } T(n) = 2^k T(n-k) + (2^k - 1) \quad \text{--- (1)}$$

$$\text{when, } \cancel{n-k} \quad n-k = 0$$

$$n = k$$

putting $k=n$ in (1), we get

$$T(n) = 2^n T(0) + (2^n - 1)$$

$$= 2^n + 2^n - 1 \quad (\because T(0) = 1)$$

$$= 2^{n+1} - 1$$

$$\therefore \text{Complexity } \therefore T(n) = O(2^n)$$

————— X —————